AWS Infrastructure Automation with Terraform and Jenkins

Portfolio Case Study | DevOps CI/CD Infrastructure Project

Owner	Deepak Kine
Cloud Region	AWS Asia Pacific (Mumbai) - ap-south-1
Core Tools	Terraform, Jenkins, GitHub, AWS EC2, S3, VPC
Outcome	Automated deploy and destroy pipeline with S3 remote state

Project Summary

This project demonstrates a complete infrastructure-as-code workflow: Terraform defines AWS resources, GitHub stores the code, Jenkins runs the CI/CD pipeline, and S3 stores Terraform state remotely.

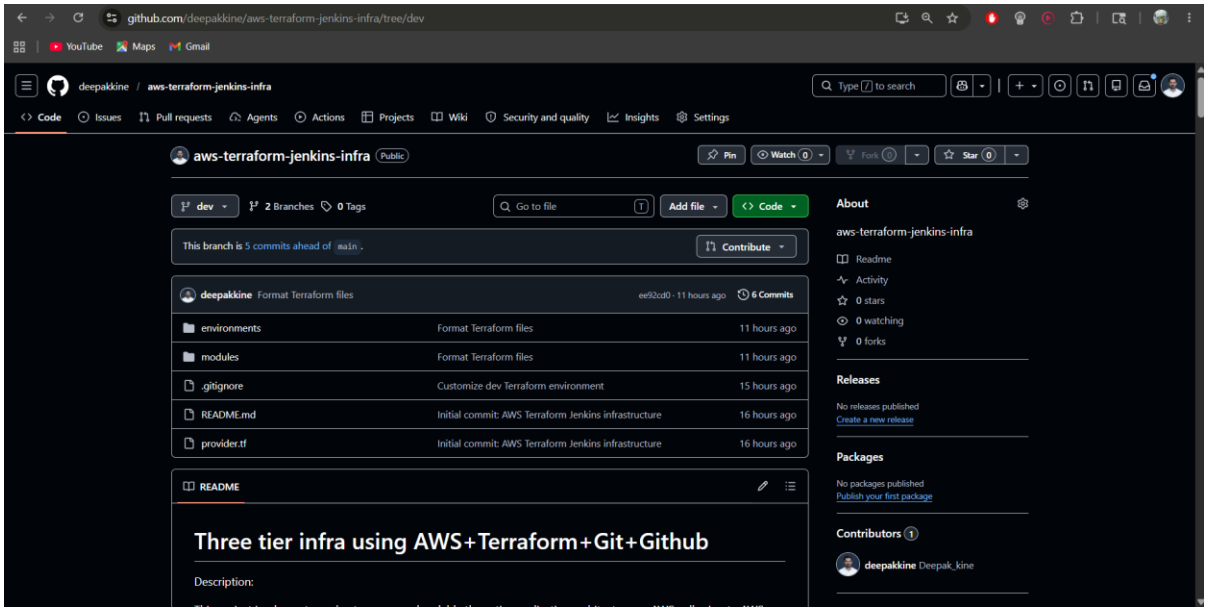
1. Project Overview

The goal of this project was to automate AWS infrastructure provisioning using Terraform and Jenkins. The final implementation uses a GitHub dev branch as the source of truth, Jenkins as the automation server, and an S3 backend for shared Terraform state.

- Created a personal GitHub repository from the cloned project and worked on a dev branch.
- Customized the Terraform configuration for a low-cost development environment.
- Configured S3 remote state so local Terraform and Jenkins use the same state file.
- Built Jenkins deployment and destroy pipelines for the full infrastructure lifecycle.
- Resolved real setup issues including provider syntax, disk space, and Jenkins temp space.

Architecture Flow

1. Code is maintained in the GitHub dev branch.
2. Jenkins pulls the latest Terraform code from GitHub.
3. Jenkins runs terraform init, fmt check, validate, plan, and apply.
4. AWS resources are created in ap-south-1.
5. Terraform state is stored in an S3 backend bucket.
6. A separate Jenkins pipeline destroys resources after testing.



GitHub repository on the dev branch used by Jenkins.

2. Terraform Customization

The cloned project was adjusted for a practical portfolio demo. The development environment was reduced to avoid unnecessary cost while still proving the workflow.

- Pinned the AWS provider to the 5.x series for predictable Terraform behavior.
- Fixed the Elastic IP syntax by replacing the older vpc argument with domain = "vpc".
- Reduced the dev environment to one EC2 instance and one S3 bucket.
- Removed NAT Gateway from dev to avoid avoidable hourly charges.

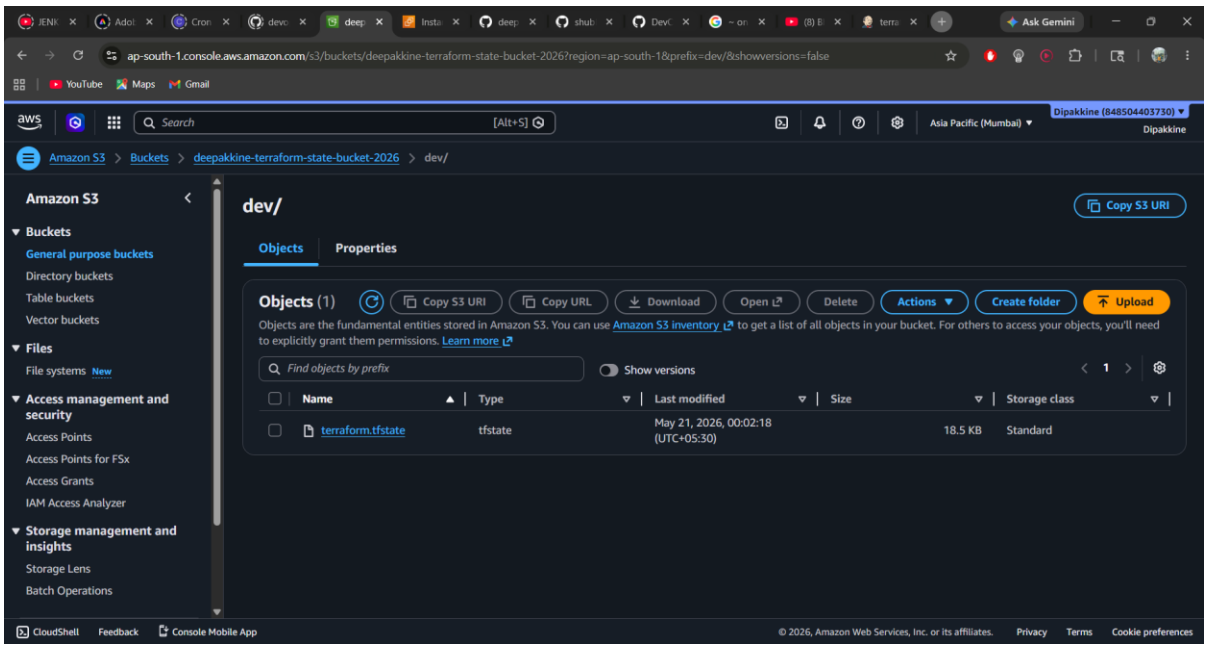
```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}
```

3. S3 Remote Backend

A dedicated S3 bucket was created to store Terraform state. This is important because Jenkins needs stable remote state instead of relying on temporary local files in the Jenkins workspace.

Backend Bucket	deepakkine-terraform-state-bucket-2026
State Key	dev/terraform.tfstate
Region	ap-south-1

```
terraform {
  backend "s3" {
    bucket  = "deepakkine-terraform-state-bucket-2026"
    key     = "dev/terraform.tfstate"
    region  = "ap-south-1"
    encrypt = true
  }
}
```



Terraform state file stored in the S3 backend bucket.

4. Jenkins Server Setup

Jenkins was installed on an AWS EC2 instance. Since the server used a free-tier-friendly instance size, swap and disk adjustments were made to keep Jenkins stable.

Instance Type	t3.micro
Storage	Expanded to 20 GiB
Access	SSH 22 and Jenkins 8080 from My IP
Runtime	OpenJDK 21, Jenkins, Terraform, AWS CLI
Stability Fixes	2 GiB swap and custom Jenkins temp directory

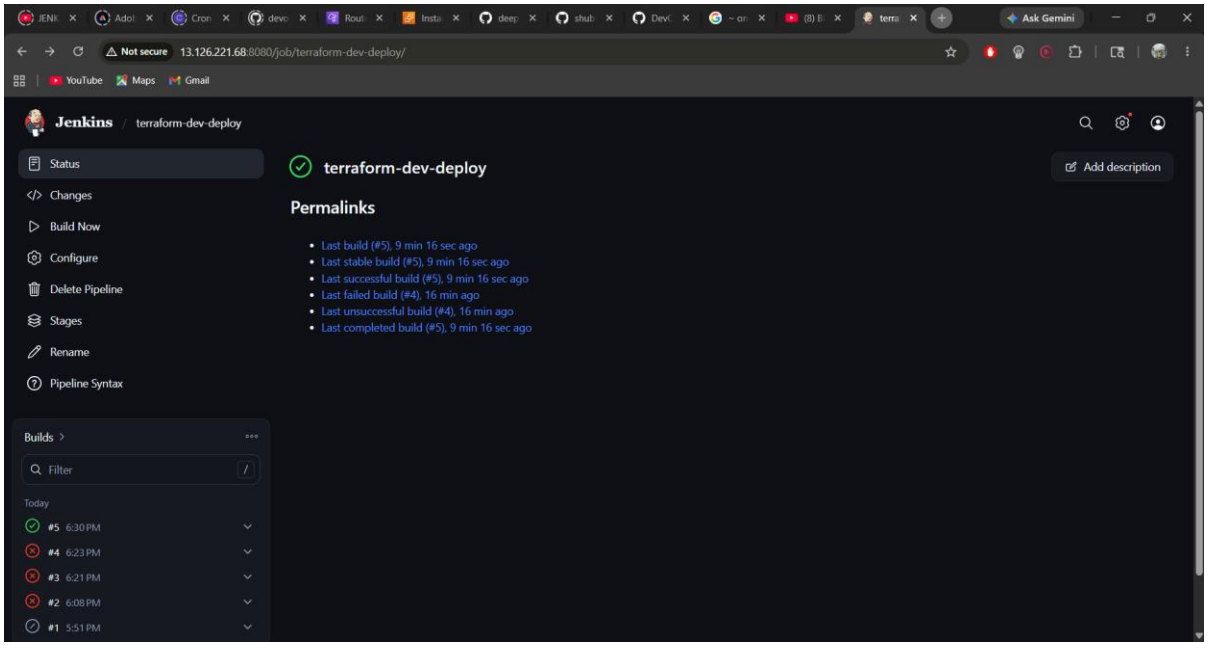
Real Troubleshooting

During setup, Jenkins marked the built-in node offline because of low disk and temp space. The root EBS volume was expanded, and Jenkins was configured to use /var/lib/jenkins/tmp.

5. Deployment Pipeline

The Jenkins deploy pipeline automates the Terraform workflow from code checkout to infrastructure creation.

Checkout Code	Pulls the dev branch from GitHub.
Terraform Init	Initializes modules, provider plugins, and S3 backend.
Terraform Format Check	Checks Terraform formatting before deployment.
Terraform Validate	Validates Terraform configuration.
Terraform Plan	Creates an execution plan.
Terraform Apply	Applies the saved plan automatically.



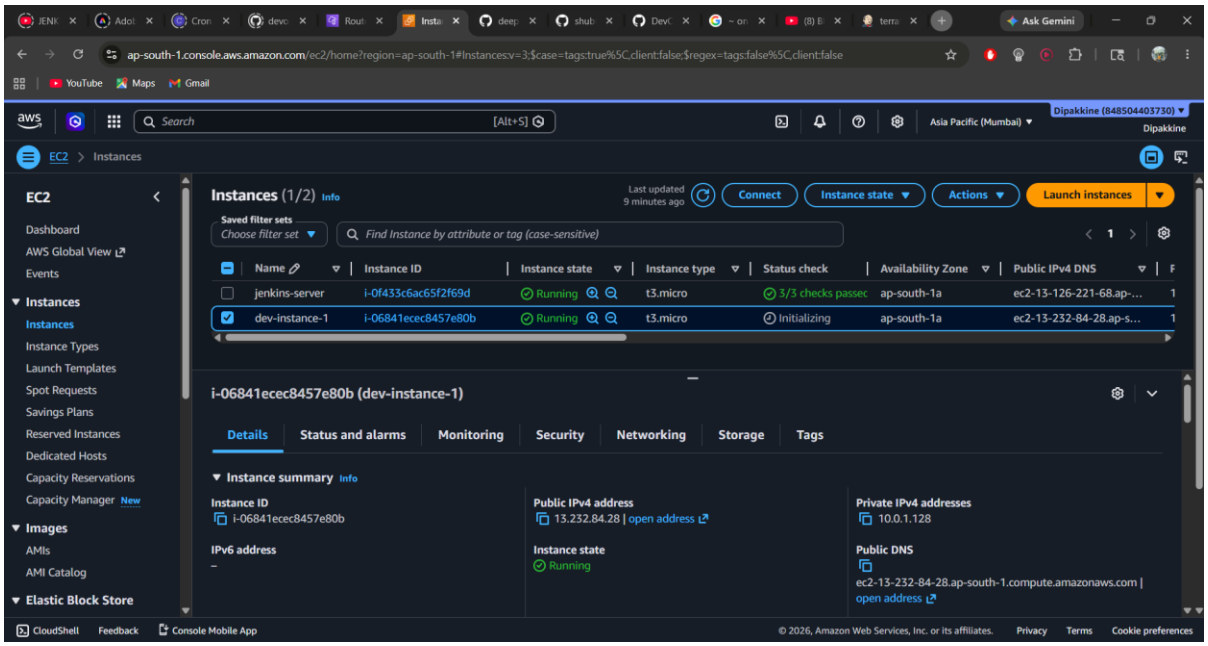
Successful Jenkins deployment pipeline.

6. AWS Resources Created

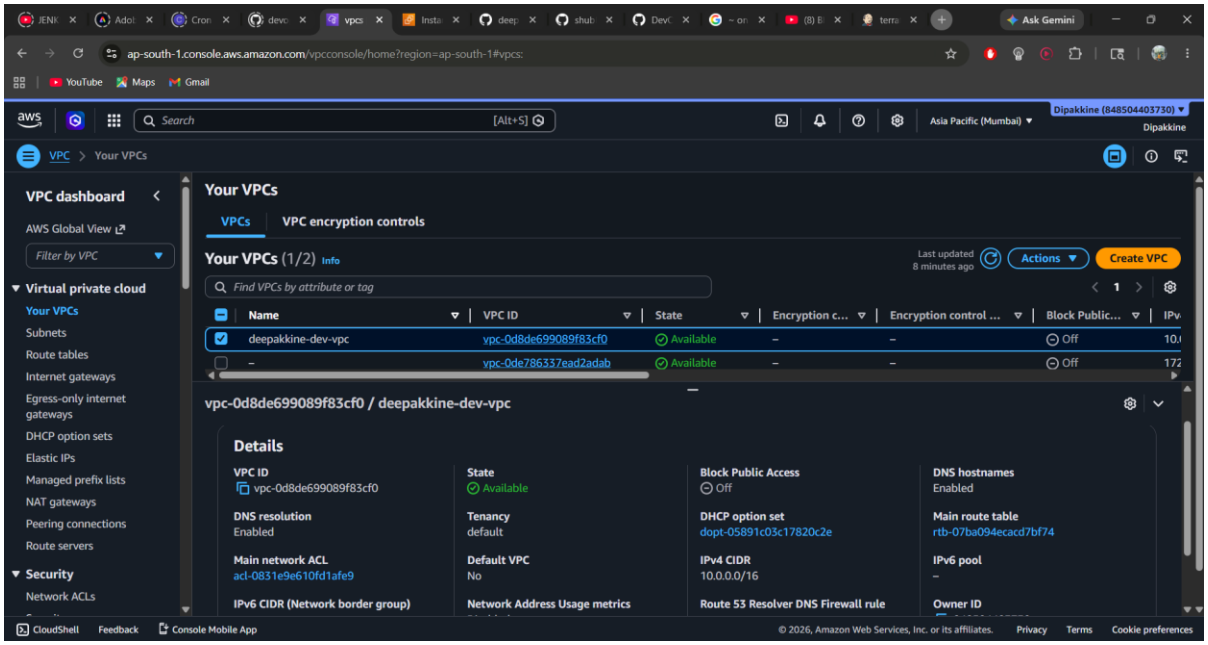
After the successful Jenkins deployment, Terraform created the development infrastructure in AWS.

EC2	One development EC2 instance.

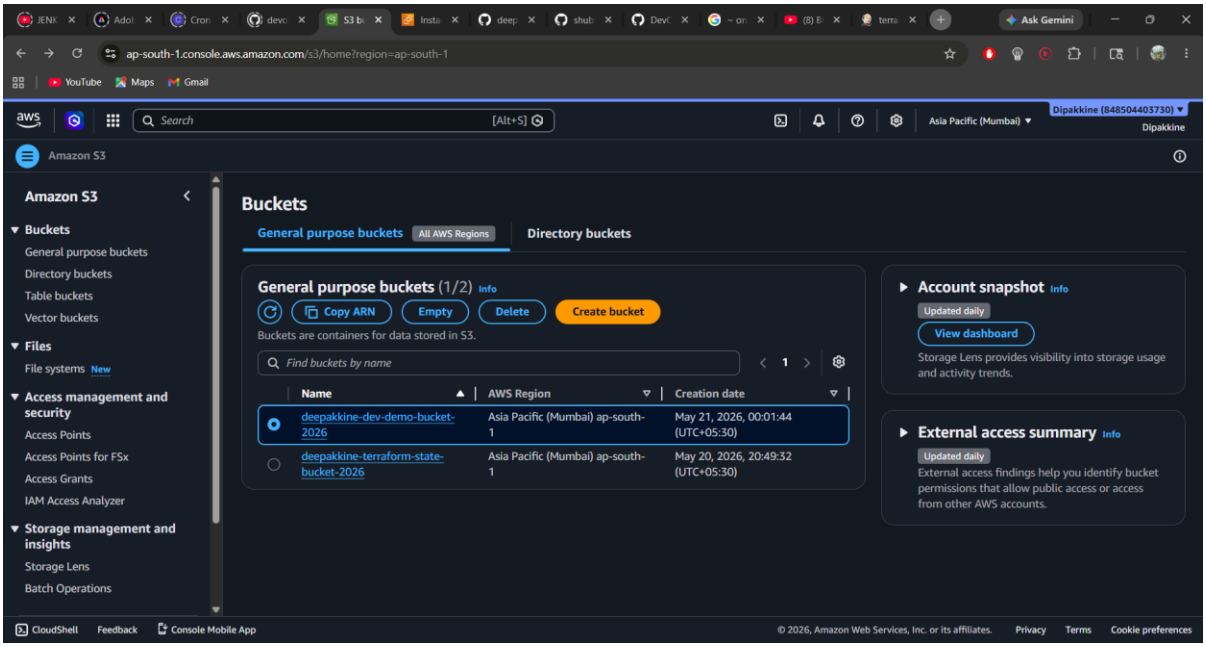
VPC	Custom development VPC.
Subnets	Public and private subnets across ap-south-1a and ap-south-1b.
Networking	Internet gateway, public route table, and route associations.
S3	Demo bucket plus Terraform backend bucket.



EC2 instance created by Terraform and visible in AWS Console.



Custom VPC created by Terraform.

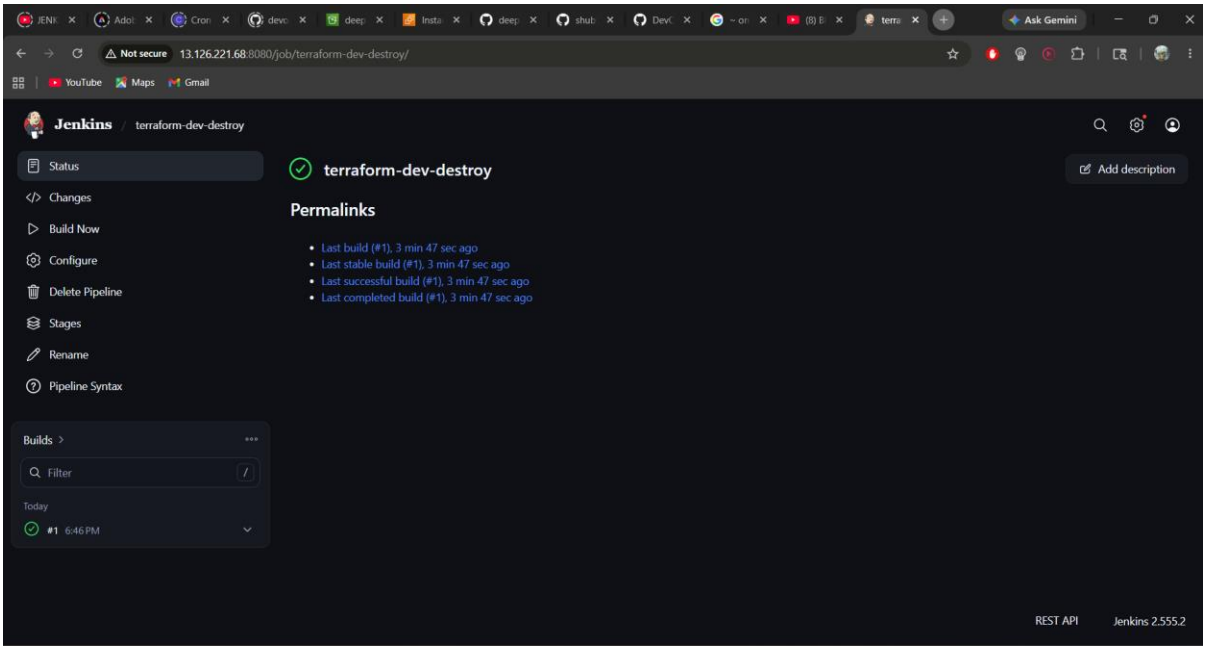


S3 buckets used by the project.

7. Destroy Pipeline and Cost Control

A separate Jenkins pipeline was created to destroy the development infrastructure after verification. This completes the lifecycle and prevents unnecessary AWS charges.

- Checks out the same dev branch.
- Initializes Terraform with the S3 backend.
- Runs terraform destroy with automatic approval.
- Leaves the backend bucket available for future state operations.



Successful Jenkins destroy pipeline.

8. Commands Used

Key commands used during the project are listed below for reproducibility.

```
# Terraform local validation
cd environments/dev
terraform init -reconfigure
terraform fmt -recursive .
terraform validate
terraform plan

# S3 backend setup
aws s3api create-bucket --bucket deepakkine-terraform-state-bucket-2026 --region ap-south-1 --create-bucket-
configuration LocationConstraint=ap-south-1
aws s3api put-bucket-versioning --bucket deepakkine-terraform-state-bucket-2026 --versioning-configuration
Status=Enabled

# Jenkins server fixes
sudo growpart /dev/nvme0n1 1
sudo resize2fs /dev/nvme0n1p1
sudo systemctl restart jenkins
```

9. Final Outcome

Result

The project successfully demonstrated CI/CD-based infrastructure automation. Jenkins deployed AWS resources using Terraform, Terraform state was stored in S3, and a destroy pipeline was used to clean up the environment after testing.

Skills Demonstrated

- Infrastructure as Code with Terraform modules and environments.
- Jenkins CI/CD pipeline creation and troubleshooting.
- AWS VPC, EC2, S3, IAM credential usage, and remote state management.
- GitHub branch workflow for automated infrastructure deployment.
- Cost-aware DevOps practices with destroy automation.